

EXPERIMENT– 3

Fundamentals of Errors and Debugging

Key Definitions

*** Error:** A mistake or blunder made by the programmer in the source code.

Example: Writing `a + b` when the intention was to multiply `a * b`.

*** Bug:** A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways.

Example: The program outputs the sum of two numbers instead of their product.

*** Debugging:** The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

Classifications of errors in C

Error type	Description	Primary cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	<code>int a = 10;</code> (missing <code>;</code>)
Compilation / Type Errors	Operations performed on incompatible types or	Assigning float/string to int pointer, using	<code>int *p = "hello";</code>

	missing declarations.	undeclared variable.	
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing library.	undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	int x = 10 / 0;
Logical Errors	Code runs to completion without crashing, but the output is incorrect.	Wrong mathematical formula, incorrect loop condition.	for(i=0;i<5;i++) instead of i<=5

Case 1 Syntax and Compilation Errors

Aim- To identify and correct a syntax error in a C program.

ERROR PROGRAM-

```
#include<stdio.h>
```

```
int main()
{
    int a = 10
    printf( "%d\n" ,a);
    return 0;
}
```

COMPILATION COMMAND&OUTPUT

```
$cc sample1.c
sample1.c:5:5: error:  ‘,’ before ‘printf’
```

ERROR:

The semicolon ; is missing after: `int a = 10`

Because of this the compiler gives an error before the `printf` statement

A terminal window with a dark background. The prompt is (kali㉿kali)-[~/Desktop]. The user enters \$ cc sample1.c. The output shows the compiler's error message: sample1.c: In function 'main': sample1.c:5:10: error: expected ',' or ';' before 'printf'. Below the error message, the code snippet is shown with a red caret and wavy line under the comma in the printf statement: printf("%d\n" , a);. The line number 5 is also visible on the left.

```
(kali㉿kali)-[~/Desktop]
$ cc sample1.c
sample1.c: In function 'main':
sample1.c:5:10: error: expected ',' or ';' before 'printf'
5 |         printf("%d\n" , a);
  |                   ^~~~~~
```

CORRECT PROGRAM

```
#include<stdio.h>
```



```
int main()
{
    int a = 10;
    printf( "%d\n" ,a);
    return 0;
}
```

OUTPUT:

10

A terminal window with a dark background. The prompt is '(kali㉿kali)-[~/Desktop]'. The user enters './a.out' and the output '10' is displayed. The prompt is shown again with a cursor.

CONCLUSION:

This is a syntax error. After adding the missing semicolon, the program compiles and runs correctly.

CASE 2: COMPILER WARNINGS (-Wall, Wextra)

Aim: To understand compiler warnings using -Wall and -Wextra.

PROGRAM:

```
#include<stdio.h>
```

```
int main()
{
    int a;
    printf( "Hello\n" );
    return 0;
}
```

STANDARD COMPILATION vs WARNING FLAGS

```
$ cc sample2.c
$/a.out
Hello

#Enabling comprehensive warning flags
$ cc -Wall -Wextra sample2.c
Sample2.c:5::9: warning: unused variable 'a' [-Wunused-variable]
```

WARNING- The variable is declared but not used in the program.

When compiled using warning flags:

```
cc -Wall -Wextra sample2.c
```

The compiler shows:

Warning: unused variable 'a'

```
(kali㉿kali)-[~/Desktop]
$ vi sample2.c

(kali㉿kali)-[~/Desktop]
$ cc sample2.c
demo1 sample2.c
(kali㉿kali)-[~/Desktop]
$ ./a.out
Hello
Experiment2
(kali㉿kali)-[~/Desktop]
$ cc -Wall -Wextra sample2.c
sample2.c: In function 'main':
sample2.c:4:13: warning: unused variable 'a' [-Wunused-variable]
    4 |         int a;
      |         ^
```

EXPLANATION

This is not a syntax error, so the program still compiles and runs. But the compiler gives a warning because an unused variable can make the program unclear or may indicate a mistake.

OUTPUT:

Hello

Conclusion:

Compiler warnings do not stop the program from running, but they help us find the program from running, but they help us find possible mistakes. Using `-Wall` and `-Wextra` is useful for writing better and cleaner C programs.

CASE 3: printf BASED DEBUGGING (Logical bug)

Aim- To detect a logical error in a C program using `()` statements.

PRAGRAM:

```
#include<stdio.h>

int main()
{
    int i, sum = 0
    for(i= 0;i<5;i++)
    {
        printf( "DEBUG: entering iteration i= %d\n" ,i);
        sum = sum + i
        printf(DEBUG: sum update to %d\n" ,sum);
    }
    Printf( "Final Sum= %d/n" ,sum);
    Return 0;
}
```

ERROR: The program is supposed to add numbers from 1 to 5, but the loop starts from 0 and runs only up to 4.

So it calculates:

$0 + 1 + 2 + 3 + 4 = 10$

Instead of

$1 + 2 + 3 + 4 + 5 = 10$

Debugging Method:

`printf()` statements are used inside the loop to display the value of `i` and `sum` after every iteration. This helps us trace where the logic is wrong.

Correct Loop:

`for(i = 1; i <= 5; i++)`

```
(kali㉿kali)-[~/Desktop]
$ gcc sample3.c -o sample3

(kali㉿kali)-[~/Desktop]
$ ./sample3
DEBUG:entering iteration i = 0
DEBUG: sum updated to 0
DEBUG:entering iteration i = 1
DEBUG: sum updated to 1
DEBUG:entering iteration i = 2
DEBUG: sum updated to 3
DEBUG:entering iteration i = 3
DEBUG: sum updated to 6
DEBUG:entering iteration i = 4
DEBUG: sum updated to 10
Final Sum = 10
```

Conclusion:

`printf()` debugging is a method to find logical errors by checking variable values during program execution.

Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

GNU: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.

GDB (GNU Debugger): The standard debugger for the GNU software system, allowing developers to inspect variable values, set execution breakpoints, and analyze crashes.

COMMNAD	SHORTHAND	DESCRIPTION
break <line / func>	B	Sets a breakpoint at a specified line number or function name.
run	R	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps over function calls).
step	s	Executes the next line of code (steps into function calls).

print<var>	p	Prints the current value of a variable or expression. Automatically prints the value of a variable after every step.
display<var>	disp	Resumes execution until the next breakpoint or program completion.
continue	c	Prints source code lines around the current execution position.
list	l	Lists all local variables and their current values in the current frame.
info locals	info loc	Displays the current call stack (chain of function calls). Displays all active breakpoints and their numbers.

backtrace	bt	Deletes the specified breakpoint number (or all if omitted).
info breakpoints	ib	Temporarily disables a breakpoint without deleting it.
delete<id>	d	Re-enables a previously disabled breakpoint.
disable<id>	dis	Exits the GDB debugging session.
enable<id>	en	
quit	q	

CASE 4: DEBUGGING A LOGICAL ERROR USING GDB

AIM: To find a logical error in a C program using GDB'

```
#include<stdio.h>           //Line1
int main()                  //Line2
{                           //Line3
    int i, sum = 0;         //Line4
    for(i = 1; i < 5; i++)  //Line5
    {                       //Line6
        sum = sum + i;      //Line7
    }                      //Line8
    printf("Sum = %d\n", sum); //Line9
    return 0;              //Line10
}
```

ERROR: The condition $i < 5$ stops before adding 5, so output becomes

Sum = 10

DEBUGGING WALK THROUGH IN GDB

```
$ cc -g sample4.c -o sample4
```

```
$ gdb ./sample4
```

```
(gdb) list
```

```
(gdb) break 7
```

Breakpoint 1 at 0x1149: file sample4.c, line 7.

(gdb) run

Starting program: ./sample4

Breakpoint 1, main () at sample4.c:7

7 sum = sum + i;

(gdb) print i

\$1 = 1

(gdb) display i

1: i = 1

(gdb) display sum

2: sum = 0

(gdb) continue

Continuing.

Breakpoint 1, main () at sample4.c:7

7 sum = sum + i;

1: i = 2

2: sum = 1

(gdb) continue

...

1:i= 4

2:sum = 6

(gdb) continue

Sum = 10

[Inferior1 (process) exited normally]

```
Reading symbols from ./sample4 ...
(gdb) list
1      #include <stdio.h>
2      int main()
3      {
4      demo1      int i, sum = 0;
5                  for(i = 1; i < 5; i++)
6                  {
7                      sum = sum + i;
8      experiment2  }
9                  printf("Sum = %d\n", sum);
10                 return 0;
(gdb) break 7
Breakpoint 1 at 0x1151: file sample4.c, line 7.
(gdb) run
Starting program: /home/kali/Desktop/sample4
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;
```



```

/          sum = sum + 1;
(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;
1: i = 2
2: sum = 1
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;
1: i = 3
2: sum = 3
(gdb) continue
Continuing.

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;
1: i = 4
2: sum = 6
(gdb)

```

```

Breakpoint 1, main () at sample4.c:7
7          sum = sum + i;
1: i = 4
2: sum = 6
(gdb) continue
Continuing.
Sum = 10
[Inferior 1 (process 45450) exited normally]
(gdb) continue
✗ The program is not being run.
(gdb)

```

CONCLUSION: GDB helped identify the logical error in the loop condition. Changing $i < 5$ to $i \leq 5$ makes the program include 5 and produced the correct output

CASE 5: DEBUGGING FUNCTION USING NEXT AND STEP

Aim:

To understand the difference between the GDB commands next and step while debugging a program containing a function call.

PROGRAM:

```
#include<stdio.h>

int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}

int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```


DESCRIPTION:

The program calculates the square of the number 5 by calling the `square()` function. Gdb provides two useful commands, `next` and `step`, to control how the function call is executed.

Option A: Using `next`

The `next` command executes the current line. If that line contains a function call, GDB executes the entire function without entering its individual statements.

```
(gdb) break main
```

```
(gdb) run
```

```
(gdb) next
```

```
(gdb) next
```

```
(gdb) print result
```

```
$1 = 25
```

```

Type apropos word to search for commands related to word
Reading symbols from ./sample5...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.c, line 11.
(gdb) run
Starting program: /home/kali/Desktop/sample5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:11
11     int result, n = 5;
(gdb) next
12     result = square(n);
(gdb) next
13     printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)

```

Here, GDB executes `square()` completely and directly returns the value 25.

Option B: Using step

The `step` command enters the called function and allows us to examine it line by line.

```

(gdb) break main
(gdb) run
(gdb) next
(gdb) step

```

GDB enters the `square()` function. We can then check the values:

```

(gdb) print x
$1 = 5

```

(gdb) next

(gdb) print result1

\$2 = 25

```
Type "apropos word" to search for commands related to this word
Reading symbols from ./sample5 ...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.c, line 11.
(gdb) run
Starting program: /home/kali/Desktop/sample5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.c:11
11      int result, n = 5;
(gdb) next
12      result = square(n);
(gdb) step
square (x=5) at sample5.c:5
5          result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
6          return result1;
(gdb) print result1
$2 = 25
(gdb) 
```

Difference:

next: Executes the function call completely without entering the function.

step: Enters the function and allows line-by-line debugging.

Conclusion:

The next command is useful when we do not need to inspect the function, while step is useful when we want to investigate the function's internal working and variable values.

CASE 6: DEBUGGING RUNTIME ERRORS USING BACKTRACE

AIM:

To identify the cause and location of a runtime error, such as division by zero, using the GDB backtrace command.

PROGRAM:

```
#include<stdio.h>

int divide(int a, int b)
{
    return a / b;    // Division by zero when b = 0
}

int calculate(int x)
{
    return divide(x, 0); // Passes 0 as divisor
}

int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
}
```

```
    return 0;  
}
```

Error:

The function calculate() calls divide() with 0 as the divisor. This causes a division-by-zero runtime error.

GDB Execution:

cc -g sample6.c -o sample6

gdb ./sample6

Inside GDB:

(gdb) run

The program reports an arithmetic exception caused by division by zero.

Now use:

(gdb) backtrace

The backtrace displays the sequence of function calls:

#0 divide(a=10, b=0)

#1 calculate(x=10)

#2 main()

```

(gdb) run sample3
Starting program: /home/kali/Desktop/sample6
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGFPE, Arithmetic exception.
0x0000555555555147 in divide (a=10, b=0) at sample6.c:4
4      return a / b;           // Division by zero when b = 0
(gdb) backtrace
#0  0x0000555555555147 in divide (a=10, b=0) at sample6.c:4
#1  0x0000555555555166 in calculate (x=10) at sample6.c:9
#2  0x000055555555517a in main () at sample6.c:14
(gdb)

```

Understanding Stack Frames:

Frame #0: divide() — the location where the error actually occurred.

Frame #1: calculate() — the function that called divide().

Frame #2: main() — the starting function that began the call sequence.

Specific frames can also be inspected:

```
(gdb) frame 0
```

```
(gdb) print b
```

```
$1 = 0
```

```
(gdb) frame 1
```

```
(gdb) print x
```

```
$2 = 10
```



```

#2  0x00005555555517a in main () at sample6.c:14
(gdb) frame 0
#0  0x000055555555147 in divide (a=10, b=0) at sample6.c:4
4      return a / b;           // Division by zero when
(gdb) print b
$1 = 0
(gdb) frame 1
#1  0x000055555555166 in calculate (x=10) at sample6.c:9
9      return divide(x, 0);    // Passes 0 as divisor
(gdb) print x
$2 = 10
(gdb)

```

This confirms that $b = 0$, causing the division-by-zero error.

Conclusion:

The GDB backtrace command helps locate the exact function where a runtime error occurs and shows the sequence of function calls that led to it. In this case, it identifies `divide()` as the source of the division-by-zero error.

CASE 7: DEBUGGING ARRAY OUT OF BOUNDS ERRORS USING GDB

AIM: To identify and correct an array out-of-bounds error in a C program using GDB.

PROGRAM:

```

#include<stdio.h>

int main()
{
    int a[5];
    int i;

```

```
for(i = 0;i <= 5;i++)  
{  
    scanf("%d",&a[i]);    // Line 8  
}  
  
return 0;  
}
```

Error:

The array `a[5]` has only 5 elements, with valid indices from 0 to 4. However, the loop allows `i` to become 5, causing the program to access `a[5]`, which is outside the array.

Inspection in GDB:

```
(gdb) break 8  
(gdb) run  
  
(gdb) print&a[0]  
$1 = (int *) 0x7fffffffd9c0  
  
(gdb) print&a[4]  
$2 = (int *) 0x7fffffffd9d0  
  
(gdb) print&a[5]
```



```
$3 = (int *) 0x7fffffffd9d4
```

```
Reading symbols from ./sample7 ...
(gdb) break 8
Breakpoint 1 at 0x114a: file sample7.c, line 8.
(gdb) run
Starting program: /home/kali/Desktop/sample7
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gn
Breakpoint 1, main () at sample7.c:8
8          scanf("%d", &a[i]);           // Line 8
(gdb) print &a[0]
$1 = (int *) 0x7ffffffdd20
(gdb) print &a[4]
$2 = (int *) 0x7ffffffdd30
(gdb) print &a[5]
$3 = (int *) 0x7ffffffdd34
(gdb)
```

The GDB inspection shows that `a[5]` is outside the valid range of the array.

Correct Loop Condition:

```
for(i = 0; i < 5; i++)
```

This allows the program to access only:

```
a[0], a[1], a[2], a[3], a[4]
```

Conclusion:

The error occurred because the loop used `i <= 5`, allowing access to `a[5]`.

Since the valid array indices are 0 to 4, changing the condition to `i < 5` prevents the out-of-bounds access.

CASE 8: DEBUGGING SEGMENTATION FAULTS (SIGSEGV) USING GDB

Aim:

To identify and understand a segmentation fault caused by accessing an invalid memory address using GDB.

Explanation:

A segmentation fault occurs when a program tries to access a memory location for which it does not have permission. A common example is trying to access memory through a NULL pointer.

PROGRAM:

```
#include<stdio.h>

int main()
{
    int *p = NULL;    // Pointer points to address 0x0
    *p = 10;          // Writing to address 0x0 causes Segmentation Fault
    printf("%d\n", *p);
    return 0;
}
```

GDB Debugging:

First, compile the program with debugging information:

cc -g sample8.c -o sample8

Start GDB:

gdb ./sample8

Run the program:

(gdb) run

GDB reports:

Program received signal SIGSEGV, Segmentation fault.

```
(gdb)
(gdb) run
Starting program: /home/kali/sample8
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x000055555555514d in main () at sample8.c:5
5          *p = 10;                // Writing to address
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
✗ Cannot access memory at address 0x0
(gdb) █
recording @q
```

The error occurs at the statement:

`*p = 10;`

Check the pointer:

(gdb) print p

\$1 = (int *) 0x0

This shows that p is a NULL pointer.

Corrected Code:

```
#include <stdio.h>

int main()
{
```



```
int x;

int *p = &x;    // Assign p to a valid memory address

*p = 10;        // Valid write

printf("%d\n", *p);

return 0;
}
```

OUTPUT:

10

```
(kali㉿kali)-[~]
$ more sample8.c
#include <stdio.h>
int main()
{
    int x;
    int *p = &x;           // Assign p to a valid memory address
    *p = 10;               // Valid write
    printf("%d\n", *p);
    return 0;
}

(kali㉿kali)-[~]
$ cc sample8.c

(kali㉿kali)-[~]
$ ./a.out
10

(kali㉿kali)-[~]
$
```

Conclusion:

The segmentation fault was caused by dereferencing a NULL pointer. GDB helped identify the invalid memory address. Assigning the pointer to a valid variable fixes the error and allows the program to run correctly.